

VUE.JS

Aula 4: Ciclo de Vida e Vite

Saindo do CDN: organizando o projeto com Single File Components

O que vamos ver hoje

1

Por que sair do CDN?

Organizando o projeto com Vite

2

Single File Components (.vue)

template, script e style em um só arquivo

3

Ciclo de vida

created, mounted e quando cada um roda

4

watch

Reagindo a mudanças de um dado específico

5

Prática

Componente .vue com ciclo de vida e watch

AULA 4 / 7

Hoje seu projeto ganha uma estrutura de verdade: arquivos separados, e você entende quando cada parte de um componente entra em ação.

Revisão rápida da Aula 3

Antes de seguir, lembre o que já sabemos fazer:

`app.component()`

Registra um componente com nome e configuração

`props`

Levam dados do pai para o filho

`emits`

Levam avisos (eventos) do filho para o pai

Fluxo de dados

Props descem, eventos sobem — nunca o contrário

Hoje: paramos de escrever tudo em um único arquivo HTML com `<script>` via CDN e organizamos o projeto em arquivos `.vue`, com uma ferramenta de build (Vite).

Por que sair do CDN?

O CDN foi ótimo para aprender sem instalar nada. Mas conforme o projeto cresce, ele começa a atrapalhar.

Limitações do CDN

- ✗ Cada componente vira um bloco dentro do mesmo app.js — sem separação real de arquivos
- ✗ Não dá para instalar pacotes do npm (ex: bibliotecas de gráficos, ícones)
- ✗ Sem recarregamento automático: você edita e recarrega a página manualmente
- ✗ Não existe um passo de build para otimizar o código para produção

Com Vite

- ✓ Cada componente vira seu próprio arquivo .vue, fácil de encontrar e editar
- ✓ Você pode instalar e importar qualquer pacote do npm
- ✓ Hot reload: a tela atualiza sozinha a cada salvamento
- ✓ Um comando gera uma versão otimizada para colocar no ar

Vite é uma ferramenta de build: ela organiza, otimiza e serve seu projeto Vue durante o desenvolvimento e na hora de publicar.

Criando um projeto com Vite

Precisa de Node.js instalado. Com ele, quatro comandos no terminal criam e ligam o projeto.

```
$ npm create vite@latest minha-app -- --template vue
$ cd minha-app
$ npm install
$ npm run dev
  → Local:    http://localhost:5173/
```

npm create vite@latest

Gera a estrutura inicial do projeto, já perguntando o template (escolhemos "vue").

npm install

Baixa as dependências do projeto (o próprio Vue incluso).

npm run dev

Sobe um servidor local com hot reload — abra o endereço mostrado no navegador.

Estrutura do projeto

O Vite já cria uma organização padrão. No começo, o que importa mora dentro de `src/`.

```
minha-app/  
├─ index.html  
├─ package.json  
└─ src/  
    ├─ main.js  
    ├─ App.vue  
    └─ components/  
        └─ TarefaItem.vue
```

main.js — cria a aplicação Vue e monta na página (o novo lugar do `createApp`)

App.vue — o componente raiz, equivalente ao antigo `<div id="app">`

components/ — pasta onde vivem os componentes filhos, um arquivo `.vue` cada

index.html — agora só tem uma `div` vazia; quem desenha tudo é o Vue

Anatomia de um Single File Component

Um arquivo .vue reúne o que antes estava espalhado: template, script e style, cada um na sua seção.

<template>

o HTML do componente

<script>

data, methods, props, emits...

<style>

CSS só desse componente (opcional: scoped)

Repare

As três seções são opcionais, mas quase todo componente tem <template> e <script>.

O nome do arquivo (ex: TarefaItem.vue) já funciona como o nome do componente — não precisa mais do app.component().

Convertendo: do CDN para um arquivo .vue

O mesmo componente tarefa-item da Aula 3, agora reorganizado em um arquivo TarefaItem.vue.

```
// app.js (CDN)
app.component("tarefa-item", {
  props: ["texto"],
  emits: ["marcar-feita"],
  template: `
    <li @click="$emit(
      'marcar-feita')">
      {{ texto }}
    </li>`
});
```

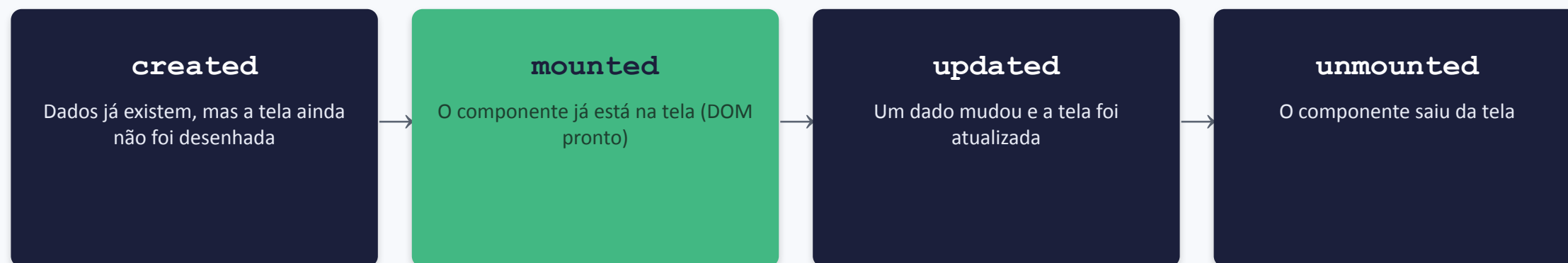
```
<!-- TarefaItem.vue -->
<template>
  <li @click="$emit('marcar-feita')">
    {{ texto }}
  </li>
</template>

<script>
export default {
  props: ["texto"],
  emits: ["marcar-feita"]
};
</script>
```

A lógica é a mesma — props e emits continuam existindo do mesmo jeito. O que muda é onde cada parte mora: o HTML vai para <template>, e a configuração do componente vira o objeto exportado em <script>.

Ciclo de vida: o que é

Todo componente passa por fases entre nascer e desaparecer da tela. O Vue avisa cada fase com um "hook".



created e mounted são os mais usados no dia a dia — vamos focar neles a seguir. **updated e unmounted** existem, mas aparecem menos no início.

Um hook é só um método com um nome especial — o Vue chama ele sozinho, no momento certo, sem você precisar pedir.

created() x mounted(): quando usar cada um

Critério	created()	mounted()
A tela (DOM) já existe?	Ainda não	Sim, já está pronta
Bom para...	Buscar dados iniciais, preparar valores	Medir elementos, integrar com bibliotecas de terceiros
Roda antes ou depois de desenhar?	Antes	Depois

Regra prática: se você só precisa preparar dados, use created(). Se precisa tocar em algo que existe na tela (um elemento, um gráfico de terceiros), use mounted().

watch: reagindo a mudanças de um dado

watch executa uma função sempre que um dado específico muda — ótimo para efeitos colaterais (side effects).

```
export default {
  props: ["filtro"],
  data() {
    return { resultados: [] };
  },
  watch: {
    filtro(novoValor, valorAntigo) {
      console.log(
        `mudou de ${valorAntigo}`,
        `para ${novoValor}`
      );
      this.buscarResultados();
    }
  }
};
```

watch x computed

computed devolve um valor calculado para ser exibido na tela.

watch não devolve nada — ele só executa uma ação (buscar dados, salvar algo, disparar um log) quando o dado observado muda.

Recebe dois argumentos: **o valor novo e o valor antigo**.

Componente TarefaItem.vue com ciclo de vida

O que construir

- Crie um projeto com Vite e um componente TarefaItem.vue
- Receba texto e feita como props
- No created(), dê um console.log("tarefa criada:", this.texto)
- No mounted(), dê um console.log("tarefa na tela")
- Use watch para observar a prop feita e logar quando ela mudar
- Emita marcar-feita ao clicar, como na Aula 3

```
<script>
export default {
  props: ["texto", "feita"],
  emits: ["marcar-feita"],
  created() {
    console.log(
      "tarefa criada:", this.texto
    );
  },
  watch: {
    feita(novoValor) {
      console.log(
        "feita agora é:", novoValor
      );
    }
  }
};
</script>
```

Recapitulando

Vite

Ferramenta de build: organiza o projeto e roda um servidor com hot reload

Single File Component

Um arquivo .vue reúne template, script e style

created()

Roda quando os dados já existem, antes de desenhar a tela

mounted()

Roda quando o componente já está na tela

watch

Executa uma ação sempre que um dado observado muda

Próxima aula: Vue Router — navegação entre páginas